



POLITECNICO
MILANO 1863

Boosting fdapde by Randomized Matrix Decompositions

Marco Galliani

February 11, 2025

Regularized fPCA

Ω spatial domain, $X = [x_i(\mathbf{p}_j)]_{i,j} \in \mathbb{R}^{N \times L}$: data matrix where x_i , are functions observed at a fixed set of points $\{\mathbf{p}_1, \dots, \mathbf{p}_L\} \subseteq \Omega$.

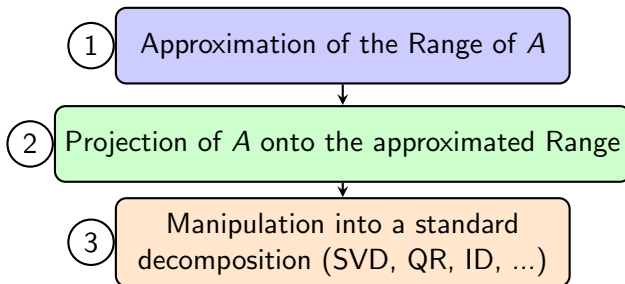
Problem (Penalized fPCA problem)

Let $F_I = [\mathbf{f}_{iI}]_{i=1}^M \in \mathbb{R}^{L \times M}$ the PCs evaluated and $S = [\mathbf{s}_i]_{i=1}^M \in \mathbb{R}^{N \times M}$ the scores, the regularized fPCA problem can be written as

$$\min_{F, S} \|X - SF_I^T\|_F^2 + \lambda \sum_{i=1}^M \mathbf{s}_i^T \mathbf{s}_i \int_{\Omega} (\Delta f_i)^2$$

$$s.t. \quad S^T S = 1$$

Given a matrix $A \in \mathbb{R}^{L \times N}$, low-rank approximations of A may be computed by:



Randomized range finder

To achieve the first step, A is iteratively applied to a random Gaussian matrix $\Omega \in \mathbb{R}^{N \times k}$

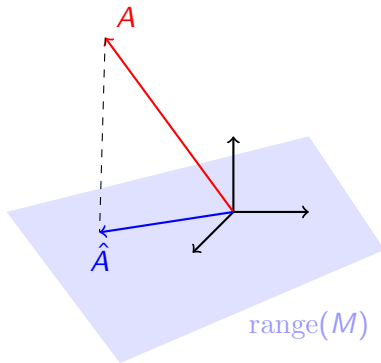
$$A\Omega, \quad (AA^T)A\Omega, \quad \dots, \quad (AA^T)^{q-1}A\Omega$$

Indeed, it can be proved that such products are strongly correlated with the k -dominant left singular vectors of A

Once we got a rank- k matrix M approximating $\text{range}(A)$, we can project the columns of A onto $\text{range}(M)$ to obtain a rank- k approximation:

$$\hat{A} = QQ^T A = QB$$

with $Q = \text{orth}(M)$, $B = Q^T A$



We can manipulate the low-rank approximation to build a truncated SVD:

Theorem (Halko et al, [2])

Given $\hat{A} = QB$ where $B = Q^T A$, such that $\|A - QB\|_F \leq \epsilon$, an approximate k -truncated SVD factorization is formed by

- 1: $[\tilde{U}, \Sigma, V] = \text{svd}(B)$
- 2: Set $U = Q\tilde{U}$

without introducing further approximation

$$\|A - U\Sigma V^T\|_F \leq \epsilon.$$

Require: A block size k , number of iterations q

RSI: $(AA^T)^{q-1}A\Omega$

- 1: Generate $Y \in \mathbb{R}^{N \times k}$
- 2: **for** $i = 1, \dots, q$ **do**
- 3: $Q = AY$
- 4: $[Q, \sim] = \text{qr}(Q)$
- 5: $Y = A^T Q$
- 6: $[Y, \sim] = \text{qr}(Y)$
- 7: **end for**
- 8: $B = Y^T$
- 9: $U\Sigma V^T = Q \text{svd}(B)$

RBKI: $[A\Omega, (AA^T)A\Omega, \dots, (AA^T)^{q-1}A\Omega]$

- 1: Generate $Y_0 \in \mathbb{R}^{N \times k}$
- 2: **for** $i = 1, \dots, q$ **do**
- 3: $Q_i = AY_{i-1}$
- 4: $Q_i = Q_i - \sum_{j < i} Q_j(Q_j^T Q_i)$
- 5: $[Q_i, \sim] = \text{qr}(Q_i)$
- 6: $Y_i = A^T Q_i$
- 7: **end for**
- 8: $B = [Y_1, \dots, Y_q]^T$
- 9: $U\Sigma V^T = [Q_1, \dots, Q_q] \text{svd}(B)$

Stopping criterion

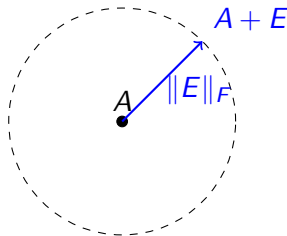
7/29

Given r rank of the truncated factorization, stop as soon as each $(\hat{\mathbf{u}}_i, \hat{\sigma}_i, \hat{\mathbf{v}}_i)$ achieve the residual accuracy:

$$r_i = (\|(A - \hat{A})^T \hat{\mathbf{u}}_i\|^2 + \|(A - \hat{A}) \hat{\mathbf{v}}_i\|^2)^{1/2} \leq \epsilon. \quad (1)$$

If (1) holds $\forall i = 1, \dots, r$, there exists a perturbation matrix E with $\|E\|_F < \epsilon$, such that

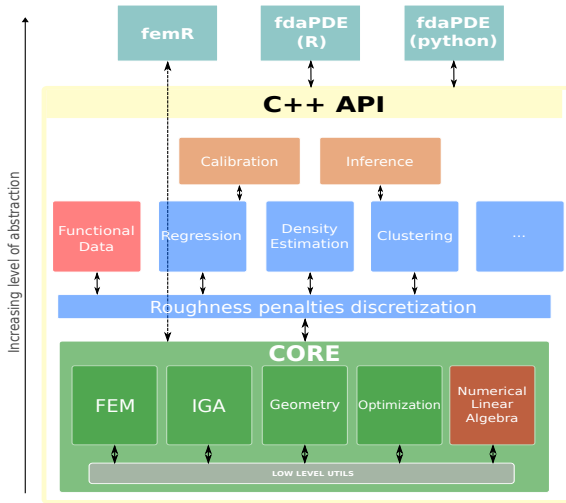
$$\text{tsvd}_r(A + E) = \hat{A}$$



Implementation - Implementing RandNLA algorithms

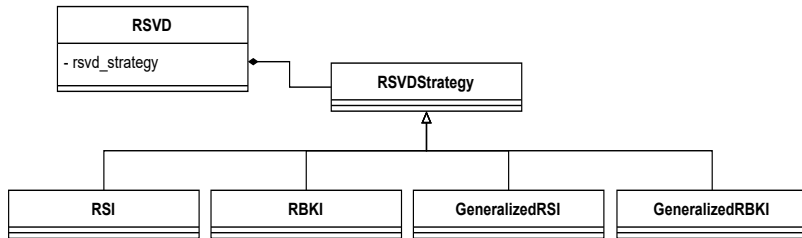
fdapde modular structure

8/29



Implementation of RandSVD

As there exists multiple methods to perform the SVD in a randomized way, the **Strategy Design Pattern** was used



RSVD class: interface with the user

```
template <typename MatrixType> class RSVD;
```

Method	Description	Parameters
compute()	Computes an SVD truncated at the rank specified by the user	Input matrix, truncation rank
		Input matrix, truncation rank, block size
matrixU() matrixV() singularValues()	Left singular vectors Right singular vectors Singular values	

RSVD: under the hood

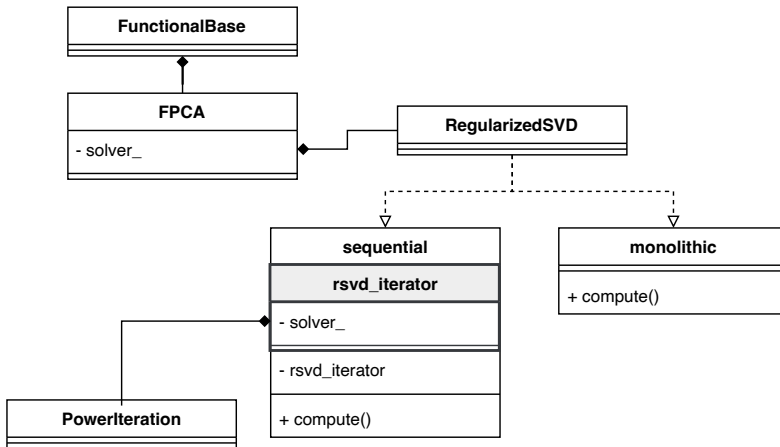
- RSVD is actually just a wrapper, all the work is delegated to the pointed RSVDStrategy

```
const DMatrix<double> matrixU() const{  
    return rsvd_strategy->matrixU();}
```

- This semantic representation allows to define a trait to detect the use of a randomized algorithms

```
template <typename T> struct is_rand_svd;
```

Current FPCA implementation



The new RegularizedSVD class

RSVD is integrated into RegularizedSVD by adding a new template parameter:

```
template <typename SolutionPolicy_, typename SVDType_>  
class RegularizedSVD;
```

Then, the SVD is dispatched using the compile time if, if constexpr:

```
if constexpr (is_rand_svd<SVDType_>{}){  
    svd_.compute(X_,rank);  
} else{  
    svd_.compute(X_,  
                 Eigen::ComputeThinU |  
                 Eigen::ComputeThinV);  
}
```

Extension to partially observed data I

Partially observed data: x_i are observed only at a subset $\mathcal{O}_i$ of the L spatial locations.

Problem (fPCA problem for partially observed data)

Let $W \in \mathbb{R}^{N \times L}$ be the binary matrix that represents the presence ($w_{ij} = 1$) or absence ($w_{ij} = 0$) of the i -th function at location j . The regularized fPCA problem translates to

$$(\hat{F}, \hat{S}) = \arg \min_{S, F} \|W * (X - SF^T \Psi^T)\|_F^2 + \lambda \operatorname{Tr}(F^T P F)$$

and $S^T S = I_M$

Extension to partially observed data II

- 1: Impute missing data to 0 and set $U_0 = \mathbf{0}_{N \times K}$ as initial guess
- 2: **for** $k = 1, \dots, r$ **do**
- 3: $i = 0$,
- 4: **while** not converged **do**
- 5: Impute the data $\tilde{X} = W * X + W^C * (U_i \Psi^T)$
- 6: Solve the fPCA problem for the imputed data

$$U_{i+1} = \arg \min_U \|\tilde{X} - U \Psi^T\|_F^2 + \lambda \text{Tr}(U P U^T)$$
$$\text{s.t. } \text{rank}(U) = k$$

- 7: $i = i + 1$
- 8: **end while**
- 9: **end for**

Implementing fPCA for partially observed data

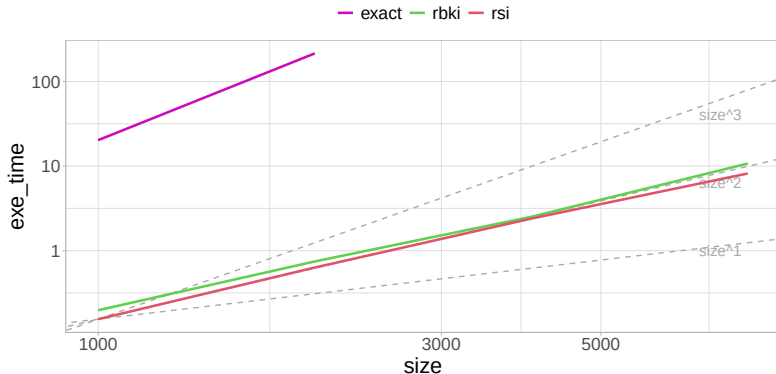
To implement the extension of the algorithm to the case of partially observed data a boolean template parameter was added to the RegularizedSVD class:

```
template <typename SolutionPolicy_, typename SVDType_,  
          bool MissingData>  
class RegularizedSVD;
```

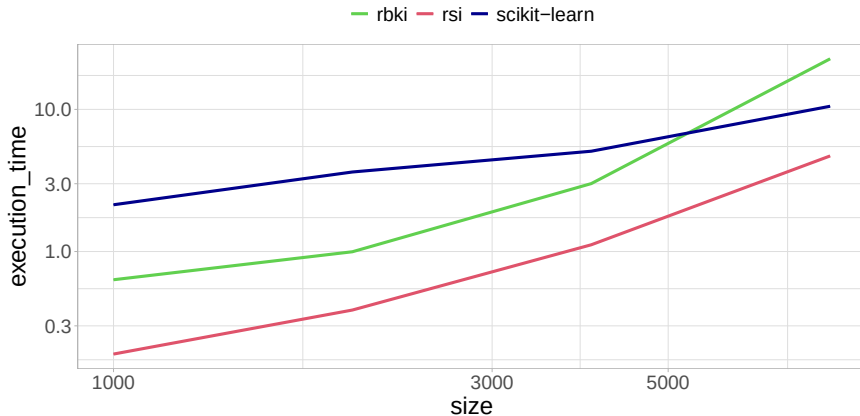
Missing data algorithms are then implemented as partial specializations of RegularizedSVD.

Test on square matrices

Test matrix $A = U\Sigma V^T$, where U, V random orthogonal matrices, Σ diagonal matrix with slow-decaying entries, $\sigma_i = \frac{1}{\log(2+i)}$.



Comparison to scikit-learn



Given $\Omega = [0, 1]^2$, the unit square domain:

1. Compute eigenfunctions of $\nabla \cdot (K \nabla u)$:

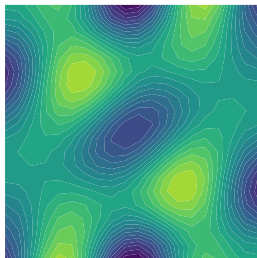
$$f_i : \Omega \rightarrow \mathbb{R}$$

2. Generate functional observations by sampling a score for each PC, $S_i \sim \mathcal{N}(0, \sigma_i^2)$, where σ_i^2 is decreasing in i .

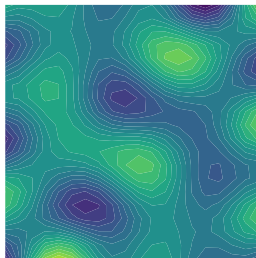
$$x_n(\mathbf{p}) = s_{1,n}f_1(\mathbf{p}) + s_{2,n}f_2(\mathbf{p}) + s_{3,n}f_3(\mathbf{p}) \quad \forall n \in 1, \dots, N$$

3. Add noise to the data:

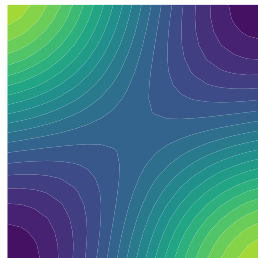
$$\tilde{X} = X + E, \quad \text{where } E \text{ Gaussian}$$



(a) PC1



(b) PC2



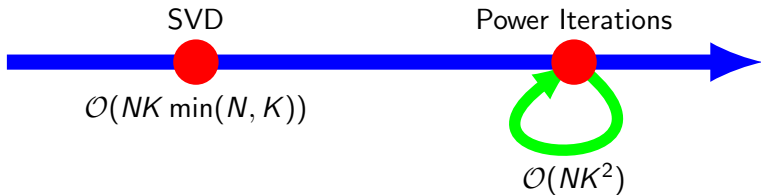
(c) PC3

N : statistical units, K : mesh nodes, sampling at mesh nodes

Monolithic fPCA



Sequential fPCA



Varying the number of statistical units I

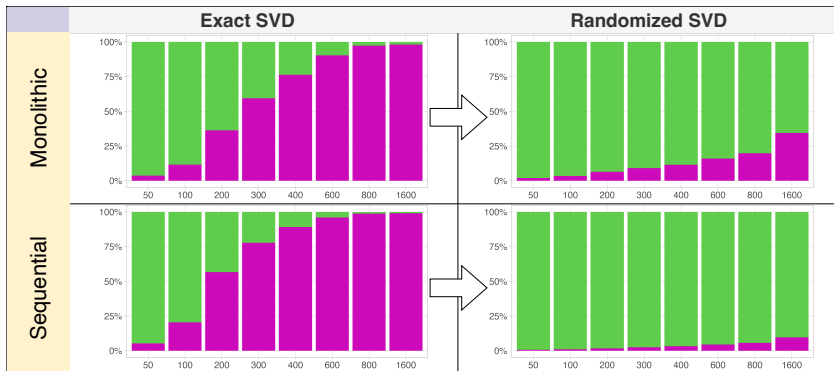
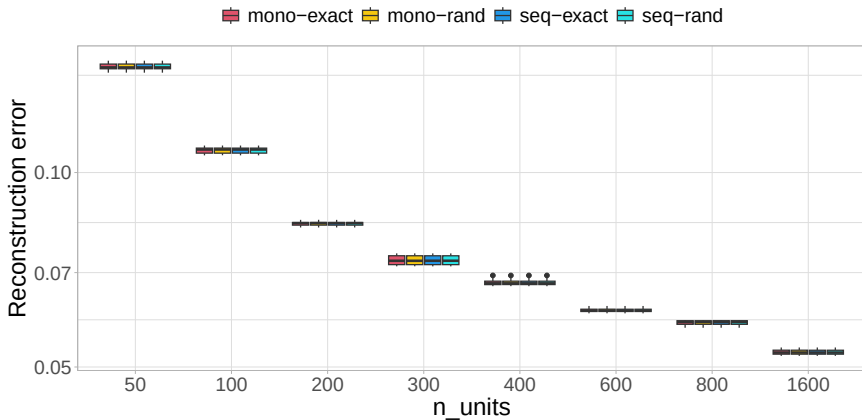


Figure: 900 mesh nodes, sampling at nodes, 3 PCs.

Varying the number of statistical units II



Varying the number of nodes in the mesh

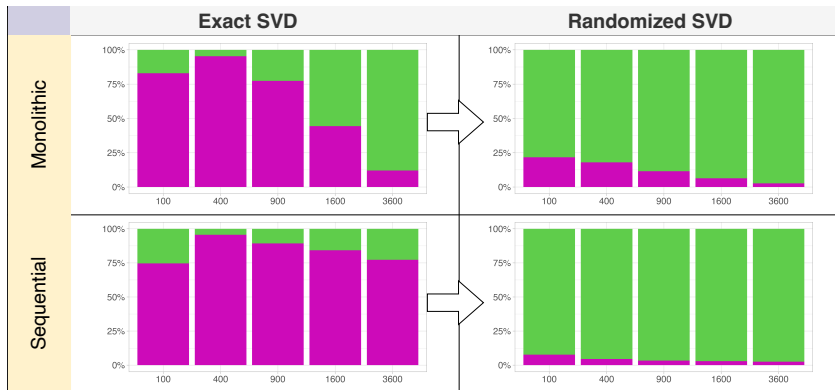


Figure: 400 statistical units, sampling at nodes, 3 PCs

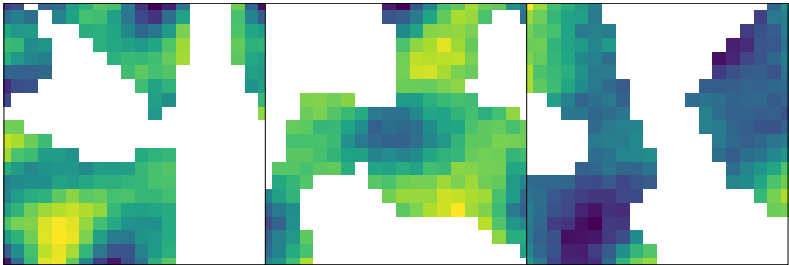


Figure: Examples of dependently censored observations

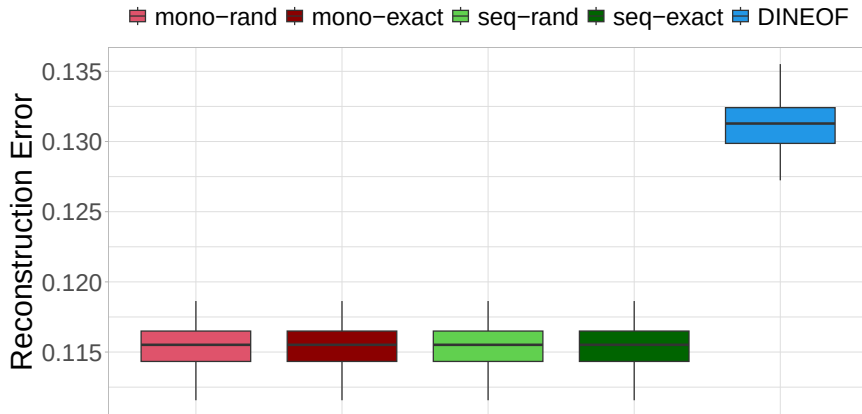


Figure: 900 mesh nodes, 400 statistical units



Figure: 900 mesh nodes, 400 statistical units



Eleonora Arnone, Aldo Clemente, Laura M. Sangalli, Eardi Lila, Jim Ramsay, and Luca Formaggia.

fdaPDE: Physics-Informed Spatial and Functional Data Analysis, 2024.

R package version 1.1-20.



Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp.

Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions, 2010.



Eardi Lila, John A. D. Aston, and Laura M. Sangalli.

Smooth principal component analysis over two-dimensional manifolds with an application to neuroimaging.

The Annals of Applied Statistics, 10(4), December 2016.



Alessandro Palummo, Eleonora Arnone, Luca Formaggia, and Laura M. Sangalli.

Functional principal component analysis for incomplete space-time data.

Environmental and Ecological Statistics, 31(2):555–582, 2024.



Joel A. Tropp and Robert J. Webber.

Randomized algorithms for low-rank matrix approximation: Design, analysis, and applications, 2023.